Day 164 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 24, Day 4 reference

Watch: p-hacking: What it is and how to avoid it! (StatQuest with Josh Starmer) | False Discovery Rates, FDR, clearly explained (StatQuest with Josh Starmer)

`day164_multiple_comparisons.py`, segment by segment, in the order `main()` uses things. The parts (segments, segment_tests, bonferroni, holm, benjamini_hochberg, METHODS) are copied unchanged into `day164_exercise.py`; Hillstrom's data and tests (Day 162) and the Adult data and logistic regression (Day 161) are imported from their exercise files. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: p = an array of p values · m = how many · alpha = 0.05 · A, B = 0/1 masks of the two groups · S = segment name → mask · FWER, FDR = family-wise error rate, false discovery rate

Segment 0 — Setup

Imports and constants

CODE

```
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day161_exercise as d161
    import day162_exercise as d162
ALPHA = 0.05
OUTCOMES = ("visit", "conversion", "spend")
S_AA = 1000
S_TRUTH = 300
FRAC = 0.3
SURE = 1e-4
N_MODELS = 100
S_SPLIT = 500
EMPTY = [0]
```

- `with contextlib.redirect_stdout(io.StringIO()):` — the two exercise files print their hand checks when imported; this swallows that output.
- `EMPTY = [0]` — a one-element list, not a number: a function can change EMPTY[0] without a global statement.

Segment 1 — The parts

segments: 22 masks in a dictionary

CODE

```
def segments(d):
    S = {}
    for c in ("channel", "zip_code"):
        for v in np.unique(d[c]):
            S[f"{c} {v}"] = d[c] == v
    S["new customer"], S["returning"] = d["newbie"] == 1, d["newbie"] == 0
    S["men's buyer only"] = (d["mens"] == 1) & (d["womens"] == 0)
    S["women's buyer only"] = (d["mens"] == 0) & (d["womens"] == 1)
    S["bought both"] = (d["mens"] == 1) & (d["womens"] == 1)
    for v in np.unique(d["history_segment"]):
        S[f"history {v[3:]}"] = d["history_segment"] == v
    for lo, hi in ((1, 3), (4, 6), (7, 9), (10, 12)):
        S[f"recency {lo}-{hi}"] = (d["recency"] >= lo) & (d["recency"] <= hi)
    return S
```

- `S[f"{c} {v}"] = d[c] == v` — an f-string builds the name; d[c] == v is a True/False array over all customers.
- `S["new customer"], S["returning"] = ...` — two dictionary entries assigned in one line from a tuple.
- `(d["mens"] == 1) & (d["womens"] == 0)` — & combines masks; the brackets are needed because & binds tighter than ==.
- `v[3:]` — drops the history band's "1) " prefix.

segment_tests: 66 tests, with a guard

CODE

```
def segment_tests(d, A, B, S):
    ps, names = [], []
    for sn, sm in S.items():
        a, b = A & sm, B & sm
        for col in OUTCOMES:
            if col == "spend" and d[col][a].max() == 0 and d[col][b].max() == 0:
                ps.append(1.0); EMPTY[0] += 1
            elif col == "spend":
                ps.append(d162.welch_test(d[col][a], d[col][b])[2])
            else:
                ps.append(d162.two_prop_test(d[col][a].sum(), a.sum(), d[col][b].sum(),
                    b.sum())[2])
            names.append(f"{sn}, {col}")
    return np.array(ps), names
```

- `a, b = A & sm, B & sm` — the segment's customers in each group.
- `d[col][a].max() == 0 and d[col][b].max() == 0` — no buyer on either side: Welch would compute 0/0, so p = 1 is used instead.
- `EMPTY[0] += 1` — counts those cases, so the run can report them.
- `[2]` — both of Day 162's tests return a tuple whose third item is the p value.

The three rules

MATH

Holm: $p_{(k)} \le \alpha/(m-k)$ with k from 0 · BH: the largest k with $p_{(k)} \le k\alpha/m$ (k from 1)

CODE

```
def bonferroni(p, alpha=ALPHA):
    return p <= alpha / len(p)
def holm(p, alpha=ALPHA):
    order = np.argsort(p); m = len(p); rej = np.zeros(m, bool)
    for k, i in enumerate(order):
        if p[i] > alpha / (m - k):
            break
        rej[i] = True
    return rej
def benjamini_hochberg(p, alpha=ALPHA):
    order = np.argsort(p); m = len(p)
    ok = p[order] <= alpha * np.arange(1, m + 1) / m
    k = np.flatnonzero(ok).max() + 1 if ok.any() else 0
    rej = np.zeros(m, bool); rej[order[:k]] = True
    return rej
```

- `np.argsort(p)` — the positions of p in increasing order: order[0] is the index of the smallest.
- `for k, i in enumerate(order):` — k counts 0, 1, 2 …; i is the original position of the k-th smallest p.
- `break` — Holm stops at the first failure: nothing after it is rejected.
- `np.flatnonzero(ok).max() + 1` — the last sorted position under BH's line, plus one: how many to reject.
- `rej[order[:k]] = True` — fancy indexing: the k smallest p values, at their original positions.

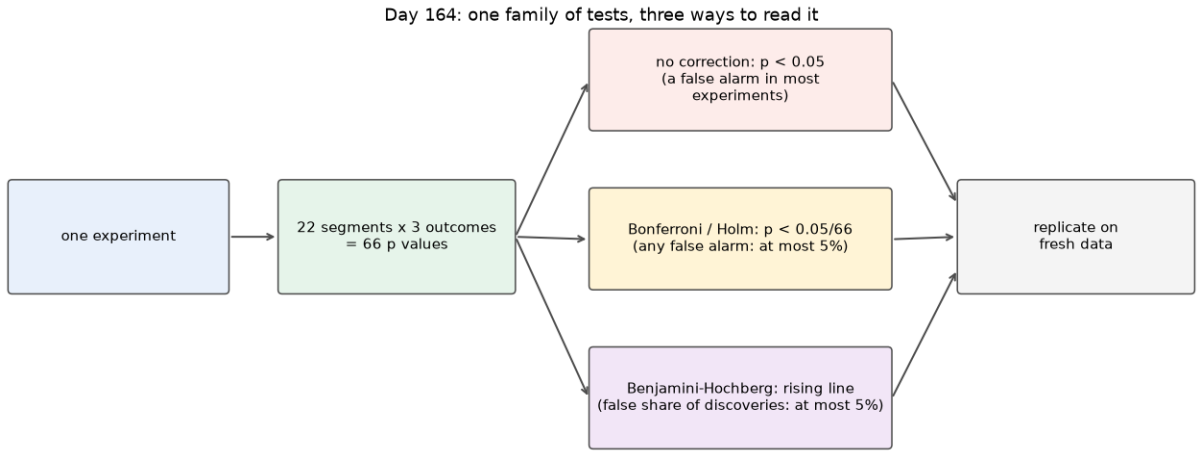
METHODS: functions as dictionary values

CODE

```
METHODS = {"no correction": lambda p: p < ALPHA, "Bonferroni": bonferroni, "Holm": holm,
           "Benjamini-Hochberg": benjamini_hochberg}
```

- `lambda p: p < ALPHA` — a one-line function, so 'no correction' can sit in the dictionary beside the others.
- `for k, f in METHODS.items():` — the experiments loop over this dictionary and call f(p) for each rule.

SYSTEM DIAGRAM



One family of tests, three ways to read it.

Segment 2 — The experiments

demo_aa: counting false alarms over 1,000 splits

CODE

```
for s in range(S_AA):
    u = rng.rand(len(c0)) < 0.5; p, _ = segment_tests(d, c0 & u, c0 & ~u, S)
    minp.append(p.min())
    for k, f in METHODS.items():
        r = f(p); anyrej[k] += r.any(); nrej[k].append(r.sum())
```

- `rng.rand(len(c0)) < 0.5` — a random True/False coin for each customer: the A/A split.
- `c0 & u, c0 & ~u` — ~ flips a boolean mask: the no-email customers with heads, and with tails.
- `r.any()` — True if the rule rejected anything; adding True to an int adds 1.

demo_truth: one family of real and null tests

CODE

```
p = np.r_[pr[real], pn]; is_real = np.r_[np.ones(real.sum(), bool), np.zeros(len(pn), bool)]
for k, f in METHODS.items():
    r = f(p); stats[k].append((np.sum(r & is_real), np.sum(r & ~is_real)))
```

- `np.r_[a, b]` — joins arrays end to end, like np.concatenate.
- `pr[real]` — boolean indexing: only the 32 tests known to be real.
- `r & is_real, r & ~is_real` — rejections that are true discoveries, and those that are false.

demo_truth: the false share in each repeat

CODE

```
fdp = np.where(tot > 0, tf[:, 1] / np.maximum(tot, 1), 0.0)
```

- `np.where(cond, x, y)` — x where cond is True, else y: a repeat with no discoveries has false share 0.
- `np.maximum(tot, 1)` — avoids dividing by zero in the branch that np.where discards.

demo_models: features to one-hot columns

CODE

```
groups = np.repeat(np.arange(12), [1] * 5 + [len(np.unique(ref[c])) for c in d161.CAT])
rng = np.random.RandomState(4); C = []; t0 = time.time()
for k in range(N_MODELS):
    use = rng.rand(12) < 0.5
    cols = np.flatnonzero(use[groups])
```

- `np.repeat(np.arange(12), counts)` — feature numbers repeated once per one-hot column: 5 numbers, then one per category.
- `use[groups]` — each column takes its feature's True/False; np.flatnonzero lists the kept columns.

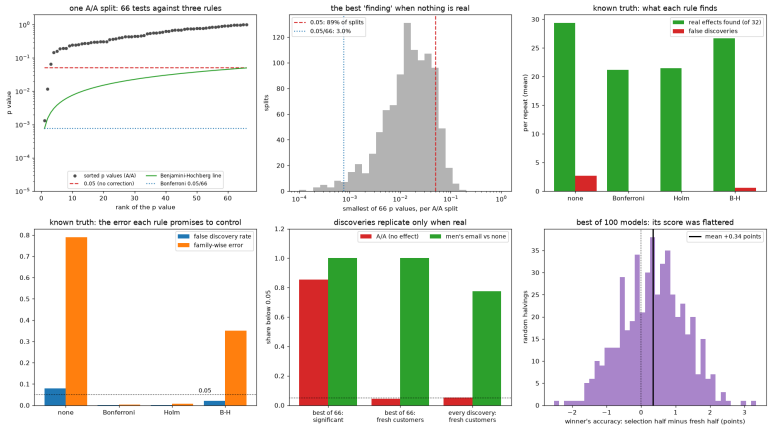
demo_models: the winner's curse in four lines

CODE

```
for s in range(S_SPLIT):
    h = rng.rand(len(va)) < 0.5
    a, b = C[:, h].mean(1), C[:, ~h].mean(1)
    w = np.argmax(a); gaps.append(a[w] - b[w]); pick.append(w)
```

- `C[:, h].mean(1)` — C is models × rows of 1/0 (correct or not): one accuracy per model on the selection half.
- `np.argmax(a)` — the index of the best model on that half; b[w] is the same model on the other half.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.argsort(p)` — indices that would sort p.
- `rej[order[:k]] = True` — set the k smallest, in place.
- `np.flatnonzero(mask)` — the positions where mask is True.
- `np.r_[a, b]` — join arrays end to end.
- `~mask` — flip a boolean array.
- `{name: function}` — functions stored in a dictionary.

End of Day 164 syntax reference. For the experiments and what they showed, see the Day 164 complete guide.